Übungsblatt 5: Der Single-Cycle-Datenpfad

Mikrocomputertechnik 1

Musterlösung

Aufgabe 1: Ein ODER-Befehl zwischen zwei Flanken (10 Punkte)

Single-Cycle-Lehrmodell: 32 Bit, kombinatorisches Lesen, Schreiben an der steigenden Flanke. Bei PC = `0x600` liegt `0x00B666B3`. Anfang: a2 = `0x28`, a1 = `0x16`, a3 = 99.

R-Felder von oben: funct7 (7), rs2 (5), rs1 (5), funct3 (3), rd (5), opcode (7 Bit). OR: funct7 = `0000000`, funct3 = `110`, opcode = `0110011`. a1 = x11, a2 = x12, a3 = x13.

1. Decodieren Sie Registerfelder und Befehl. (3 P)
2. Geben Sie die Registeradressports A1, A2, A3 und Datenports RD1, RD2, WD vor der Flanke an. (3 P)
3. Welche Werte haben a3 und PC vor beziehungsweise nach der Flanke? (2 P)
4. Nur RegWrite wird irrtümlich null. Ändert das die ALU-Berechnung oder die gespeicherte Wirkung? (2 P)

Musterlösung

1. rs1 = 12, rs2 = 11, rd = 13; `or a3, a2, a1`. Register zusammen 2 P, Operation 1 P.
2. A1 = 12, A2 = 11, A3 = 13; RD1 = 40, RD2 = 22, WD = `0x3E` = 62. Je 0,5 P.
3. Vorher a3 = 99 und PC = `0x600`, danach a3 = 62 und PC = `0x604`. Je Zustandspaar 1 P.
4. Das kombinatorische Ergebnis 62 bleibt berechnet, aber a3 bleibt mangels Übernahme 99. Je 1 P. Der PC kann regulär weiterlaufen.

Aufgabe 2: Ein Wort im Paket versetzen (12 Punkte)

RV32I, Little Endian, gültige wortausgerichtete Zugriffe. s2 = `0x820C`. Die Wörter ab `0x8200` lauten $[-24, 17, 0, \texttt{0x11223344}]$. a2 ist anfangs 55. Erster Befehl bei `0x680`.

```
lw a2, -12(s2)
sw a2,  -4(s2)
```

1. Geben Sie die zwölf Offsetbits und das 32-Bit-ImmGen-Ergebnis des Loads an. (2 P)
2. Bestimmen Sie für beide Befehle Adresse, ALU-Funktion, zweite ALU-Quelle und Schreibziel. (4 P)
3. Welche Register- und Paketwerte ändern sich, welche bleiben erhalten? (3 P)
4. Nennen Sie die vier Bytes des geänderten Worts und den PC nach beiden Befehlen. (2 P)
5. Wieso bleiben die Store-Daten verfügbar, obwohl die ALU das Immediate benutzt? (1 P)

Musterlösung

1. −12: `1111 1111 0100`, ImmGen = `0xFFFFFFF4`. Je 1 P.
2. lw: Adresse `0x8200`, ADD, Immediate −12, Register a2 aus Speicher. sw: Adresse `0x8208`, ADD, Immediate −4, Speicher aus RD2 = a2. Je Befehl 2 P.
3. a2 wird −24 (1 P); Paket danach $[-24, 17, -24, \texttt{0x11223344}]$ (1 P); s2 bleibt `0x820C` und sw verändert a2 nicht (1 P).
4. Bytes `E8 FF FF FF`; PC nach beiden Befehlen `0x688`. Je 1 P.
5. RD2 verzweigt zusätzlich zum Speicher-Schreibdatenport. ALUSrc wählt nur den zweiten ALU-Operanden. 1 P.

Aufgabe 3: Ein Rückwärtssprung (12 Punkte)

Ein beq liegt bei PC = `0x740`, Ziel = `0x72C`. Fall A: Quellen 9 und 9. Fall B: Quellen 9 und 12. ImmGen liefert fertige signed Byteoffsets einschließlich Bit 0.

1. Berechnen Sie Offset und beide PC-Kandidaten. (3 P)
2. Bestimmen Sie je Fall 32-Bit-ALU-Differenz, Branch, Zero, PCSrc und PC danach. (4 P)
3. Ein anderer Branch bei demselben PC hat Offset +28. Nennen Sie ImmGen-Wort und Ziel. (2 P)
4. Welche falsche Adresse ergäbe sich im ersten Fall bei erneutem Verdoppeln des Offsets? (1 P)
5. Statt beq wird bei PC = `0x740` ein AND mit Quellen `0x50` und `0x0F` ausgeführt. Geben Sie Zero, PCSrc und nächsten PC an. (2 P)

Musterlösung

1. Offset −20 Byte; Kandidaten `0x744` und `0x72C`. Je 1 P.

Fall	ALU-Differenz	Branch	Zero	PCSrc	PC danach
A	`0x00000000`	1	1	1	`0x72C`
B	`0xFFFFFFFD`	1	0	0	`0x744`

2. Je Fall 2 P, je Tabellenangabe 0,4 P.
3. ImmGen = `0x0000001C`; Ziel `0x75C`. Je 1 P.
4. $0x740 - 40 = 0x718$; der fertige Offset darf nicht erneut verschoben werden. 1 P.
5. Ergebnis null, also Zero = 1; Branch = 0, damit PCSrc = 0 und nächster PC = `0x744`. Zero 0,5 P, Auswahl 0,5 P, PC 1 P.

Aufgabe 4: Steuern und einen falschen Store erklären (8 Punkte)

Kurskonvention: ALUSrc 0 = RD2, 1 = Immediate; MemtoReg 0 = ALU, 1 = Speicher.

1. Erstellen Sie für or, sub, lw, sw und beq eine Tabelle mit ALUSrc, MemtoReg, RegWrite, MemRead, MemWrite, Branch und ALU-Funktion. Markieren Sie unerhebliche Auswahlen mit X. (5 P)
2. `sw a2, -4(s2)` startet mit a2 = 12 und s2 = `0x820C`. Nur ALUSrc steht fälschlich auf 0. Der Bereich `0x8200`–`0x823F` ist beschreibbar. Welche Adresse und Daten werden tatsächlich geschrieben? (2 P)
3. Warum kann MemtoReg beim korrekt gesteuerten Store unerheblich sein, MemWrite aber nicht? (1 P)

Musterlösung

Befehl	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALU
or	0	0	1	0	0	0	OR
sub	0	0	1	0	0	0	SUB
lw	1	1	1	1	0	0	ADD
sw	1	X	0	0	1	0	ADD
beq	0	X	0	0	0	1	SUB

1. Je Zeile 1 P, anteilig je korrektes Feld 1/7 P. Bei MemtoReg für sw und beq sind begründete konkrete Werte ebenfalls zulässig.
2. Falsche Adresse $\texttt{0x820C} + 12 = \texttt{0x8218}$; Datenwort 12. Richtige Adresse wäre `0x8208`. Adresse und Daten je 1 P.
3. RegWrite = 0 sperrt die Registerübernahme; MemWrite entscheidet hingegen unmittelbar über die Store-Wirkung. 1 P.

Aufgabe 5: Den gemeinsamen Takt festlegen (8 Punkte)

Didaktische Verzögerungen in ns, keine Messwerte:

Baustein	Zeit
Clock-to-Q	0,15
Befehlsspeicher	1,60
Registerlesen	0,90
ALU	1,10
Datenspeicherlesen	2,40
Rückschreib-MUX	0,25
Setup	0,20

Alle anderen benötigten Pfade sind nicht länger als der Loadpfad; Operanden und Steuerung sind rechtzeitig. Skew und Reserve werden vernachlässigt.

1. Zeichnen Sie die Bausteinfolge von OR- und Load-Rückschreibweg und berechnen Sie die jeweilige Mindestzeit. (4 P)
2. Bestimmen Sie gemeinsame Taktperiode und ideale Maximalfrequenz. (2 P)
3. Welche Periode erhält OR in diesem Entwurf? Bei einem unabhängigen Branch erreichen Ziel und Vergleich den Auswahlbaustein 3,8 ns und 4,1 ns nach der Flanke; Auswahl und PC-Setup benötigen zusammen 0,5 ns. Welche Mindestzeit verlangt dieser PC-Pfad? (2 P)

Musterlösung

1. OR: PC → Befehlsspeicher → Registerlesen → ALU → Rückschreib-MUX → Register. $T_{\text{OR}} = 0{,}15 + 1{,}60 + 0{,}90 + 1{,}10 + 0{,}25 + 0{,}20 = 4{,}20$ ns. Load zusätzlich mit Datenspeicher zwischen ALU und Rückschreib-MUX: $T_{\text{lw}} = 6{,}60$ ns. Je Pfadfolge und Zeit 1 P.
2. $T_{\text{clk,min}} = 6{,}60$ ns; $f_{\text{max}} \approx 151{,}5$ MHz. Je 1 P.
3. Auch OR erhält mindestens 6,60 ns (1 P). Der Branchpfad verlangt $\max(3{,}8, 4{,}1) + 0{,}5 = 4{,}6$ ns, weil die beiden Eingangswege parallel laufen (1 P).

Aufgabe 6: Ein neuer Pakettrace in Ripes

1. Laden Sie `beispiele/lab_05_paketpfad.s` in einen RV32-Single-Cycle-Prozessor ohne C. Notieren Sie Version und Modell.
2. Halten Sie jeweils vor `trace_or`, `trace_load` und `trace_store`. Sagen Sie Registeradressports, Datenquellen, ALU-Ergebnis, Schreibquelle und Freigaben voraus. Führen Sie dann genau eine Clock aus und vergleichen Sie.
3. Lesen Sie die tatsächliche Basis B von paket ab. Prüfen Sie die vier Wörter und die geänderten Bytes nach dem Store.
4. Untersuchen Sie `trace_beq` zunächst mit t0 = −24, dann nach Neustart mit t0 = −23. Notieren Sie tatsächliche Zieladressen und den Endwert a4 bei inspect.
5. Ordnen Sie Funktionen den Kurs-Signalen zu. Übernehmen Sie MUX-Portnummern Ihrer Ripes-Version nicht ungeprüft.

Setup und abschließende Inspektionsschleife gehören nicht zum untersuchten Datenpfad-Ausschnitt.

Musterlösung

OR: a3 wird 62. Load: Adresse B, a2 wird −24. Store: Adresse B + 8, Daten −24; Paket danach $[-24, 17, -24, \texttt{0x11223344}]$, geänderte Bytes `E8 FF FF FF`.

Branch: t0 = −24 führt zu genommenem Branch und a4 = −24; t0 = −23 führt über den Nichtgleichheitspfad zu a4 = 0. inspect hält die Werte über einen wirkungslosen beq-Sprung fest. Das sind Modellvorhersagen; tatsächliche Portnamen und Anzeigen sind selbst zu dokumentieren.